



TCC Pantry

An offline-first office snacks & drinks ordering app —
Flutter on every screen, Cloudflare Workers at the edge.

CLIENT	BACKEND	DATABASE	VERSION
Flutter • Dart 3.11	Hono + Workers	Neon Postgres	1.5.2

OVERVIEW

01 What is TCC Pantry?

TCC Pantry is a cross-platform application that runs the daily office snack round. Employees order snacks, vote on the drink of the day, and review their history; admins manage the catalogue, user roles, holidays, and get a live order summary they can share straight to WhatsApp. It ships from a single Flutter codebase to iOS, Android, web, and desktop, backed by a serverless API deployed to Cloudflare's edge.

4

Target platforms

5

API route groups

100%

Offline-capable orders

SSO

Microsoft Entra ID

What it does

■ Order snacks

Browse a live catalogue in a tactile 3-column grid with search, veg filter, and categories. Selections are confirmed against a daily cutoff time.

■ Vote on drinks

A dedicated drinks tab for the day's beverage vote, including a per-drink sugar-free preference that is snapshotted into history.

■ Review & re-order

The last seven days of orders, grouped by date, with one-tap re-ordering that resolves items by id and falls back to name.

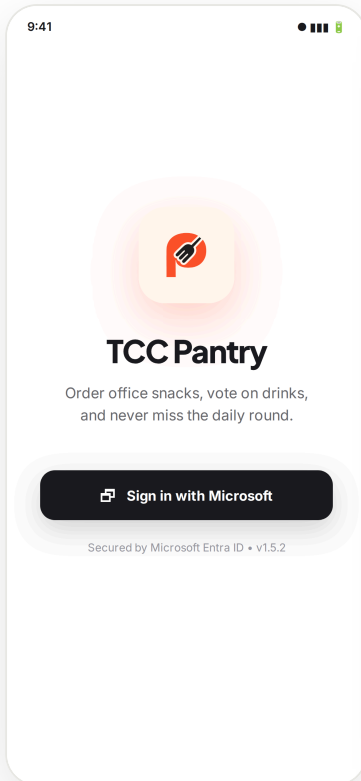
■ Admin & summary

Manage snacks, users, holidays and shutdown days; see a live count of who has ordered and export the summary to WhatsApp.

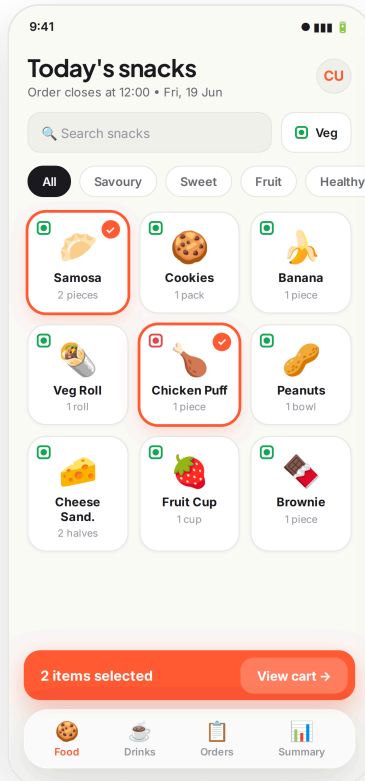
Design principle — offline first. Placing an order or voting never blocks on the network. Mutations are written to a local SQLite queue and optimistically reflected in the UI; a sync engine replays them when connectivity returns and drops anything the server rejects with a 4xx.

02 The app, screen by screen

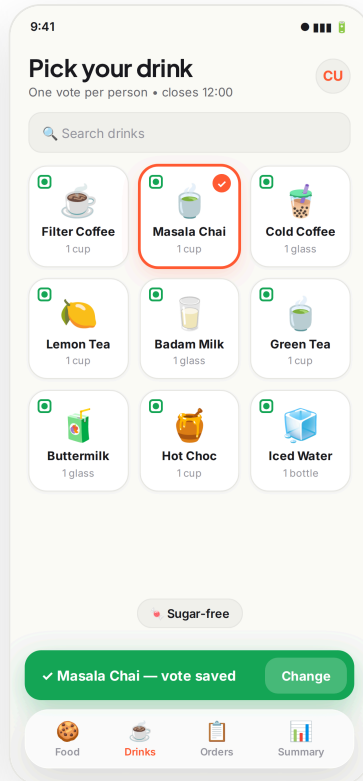
High-fidelity renders of the core flows, built from the app's own design tokens — the coral `#FF5A33` brand accent, Plus Jakarta Sans / Inter type pairing, and the frosted-glass navigation that defines the Notion-inspired theme.



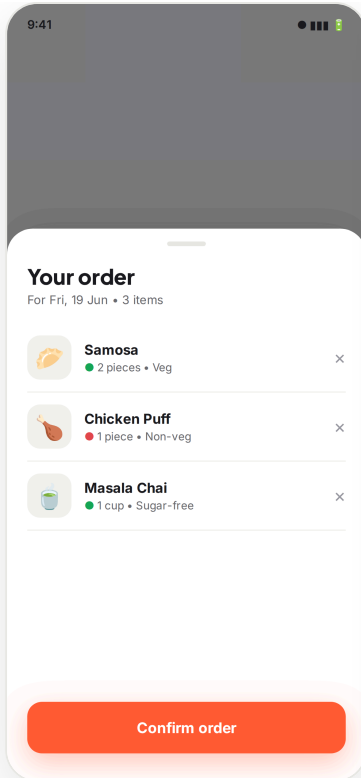
Onboarding — Microsoft sign-in



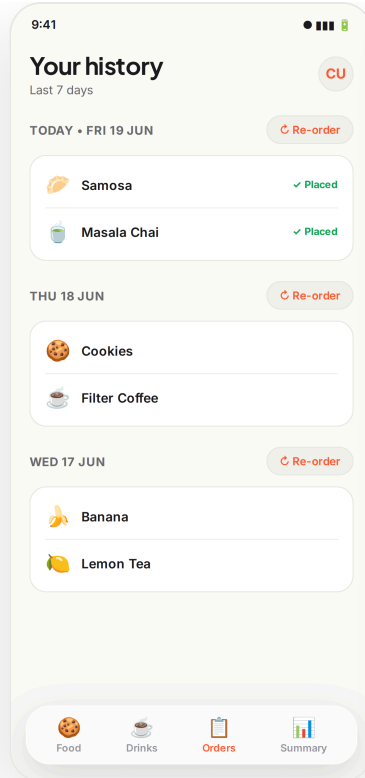
Food tab — snack grid & cart



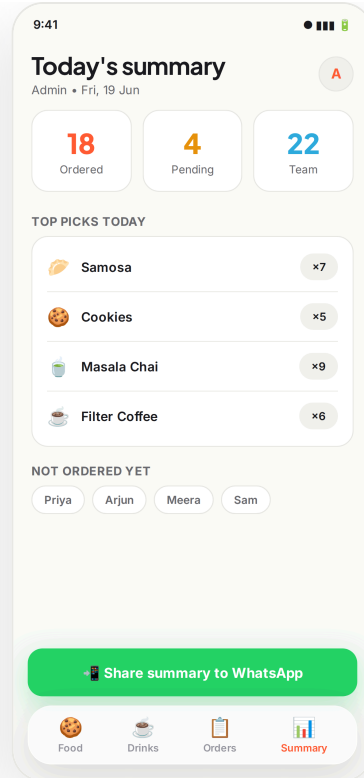
Drinks tab — daily vote



Cart sheet — confirm order



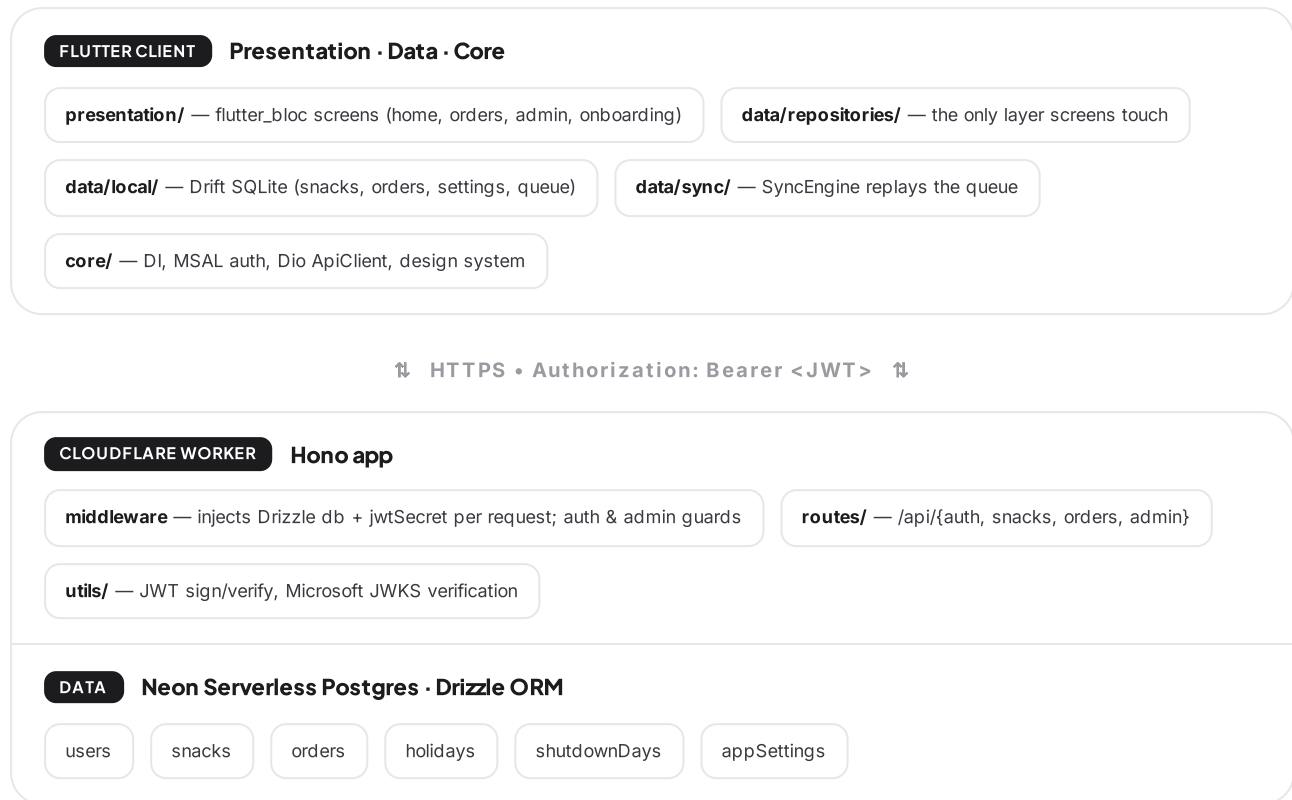
History — last 7 days



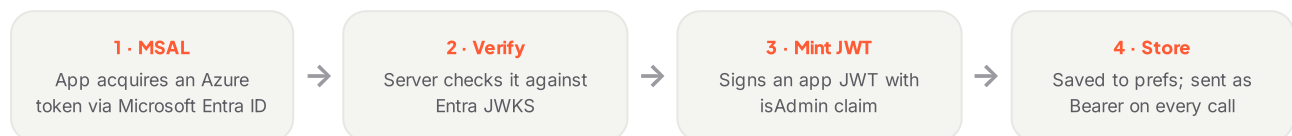
Admin — live summary

03 How it fits together

A layered Flutter client wired through `get_it` talks to a single Hono app on Cloudflare Workers over HTTPS. The client owns a local SQLite database (Drift) for offline reads and a sync queue for offline writes; the server owns a Neon Postgres database accessed through Drizzle.



Authentication flow



The router redirect gates the whole app behind the stored token — no token routes to onboarding, a valid token to home. A 401 from the API triggers a silent MSAL re-auth to refresh the JWT.

04 Data & the offline engine

Server — Postgres (Drizzle)

Table	Purpose
<code>users</code>	Identity, Microsoft id, isAdmin
<code>snacks</code>	Catalogue: veg, default, active, serving
<code>orders</code>	Per-user, per-date items + snapshots
<code>holidays</code>	Manual / auto no-order days
<code>shutdownDays</code>	Kitchen-closed days
<code>appSettings</code>	Cutoff time, advance-order window

Client — SQLite (Drift)

Table	Purpose
<code>LocalSnacks</code>	Cached catalogue for offline browse
<code>LocalOrders</code>	Optimistic orders w/ name snapshot
<code>LocalSettings</code>	Cutoff & advance-order config
<code>SyncQueue</code>	Pending mutations to replay

The offline-first write path



Why snapshots? Each order stores the snack's name and emoji at the time it was placed (`snackNameSnapshot`). History stays correct even if an admin later renames or deletes the snack, and re-order can still resolve items by name.

Smart behaviours

- **Advance-order mode** — admins can shift ordering to the next day within a configurable time window; the effective order date becomes tomorrow.
- **Share counts** — a snack can “serve 2”, and the admin summary adjusts its counts accordingly.
- **Holiday / shutdown gating** — on load the app checks `/api/orders/status` ; on a holiday the ordering UI is replaced with a friendly closed state.

05 API & technology

HTTP API

Method & path	Auth	Purpose
POST /api/auth/microsoft	public	Verify MSAL token, mint app JWT
GET /api/auth/session	jwt	Refresh session & token
GET /api/snacks	public	Active snack catalogue
GET /api/snacks/settings	public	Cutoff time & advance-order config
GET /api/orders/status	jwt	Is today a holiday / shutdown?
GET /api/orders/today	jwt	The user's order for today
POST /api/orders	jwt	Place or update today's order
DELETE /api/orders	jwt	Clear today's order
GET /api/orders/history	jwt	Last 7 days of orders
GET/POST/PUT/DELETE /api/admin/snacks	admin	Catalogue CRUD + bulk import
GET /api/admin/summary	admin	Today's counts & who has ordered
GET/POST /api/admin/users	admin	List users, toggle admin flag
GET/POST/DELETE /api/admin/holidays	admin	Manage holidays & shutdown days

Client stack

Package	Role
flutter_bloc	State management
drift + sqlite3	Local database
dio	HTTP client
go_router	Routing & auth gate
msal_auth	Microsoft Entra sign-in
get_it	Dependency injection
connectivity_plus	Reconnect detection
freezed	Immutable models

Server stack

Package	Role
hono	Web framework
drizzle-orm	Type-safe ORM
@neondatabase/serverless	Postgres driver
jose	JWT sign / verify
wrangler	Workers tooling
Bun	Runtime & test runner

Secrets — DATABASE_URL , JWT_SECRET ,
AZURE_TENANT_ID , AZURE_CLIENT_ID via wrangler.